

GPU

Part 1 - Architecture Review

Workshop

April 2024

Table of Content

What is GPU?

Quick overview of GPUs

1

GPU Hardware Model

Discuss GPU Architecture

2

Introduction to CUDA

Despite being red, it's cold

3

What is GPU

GPUs or **Graphics Processing Units** are electronic circuits that are specialized in executing **embarrassingly parallel workloads**.

Initially designed to accelerate **computer graphics** and **image processing**.



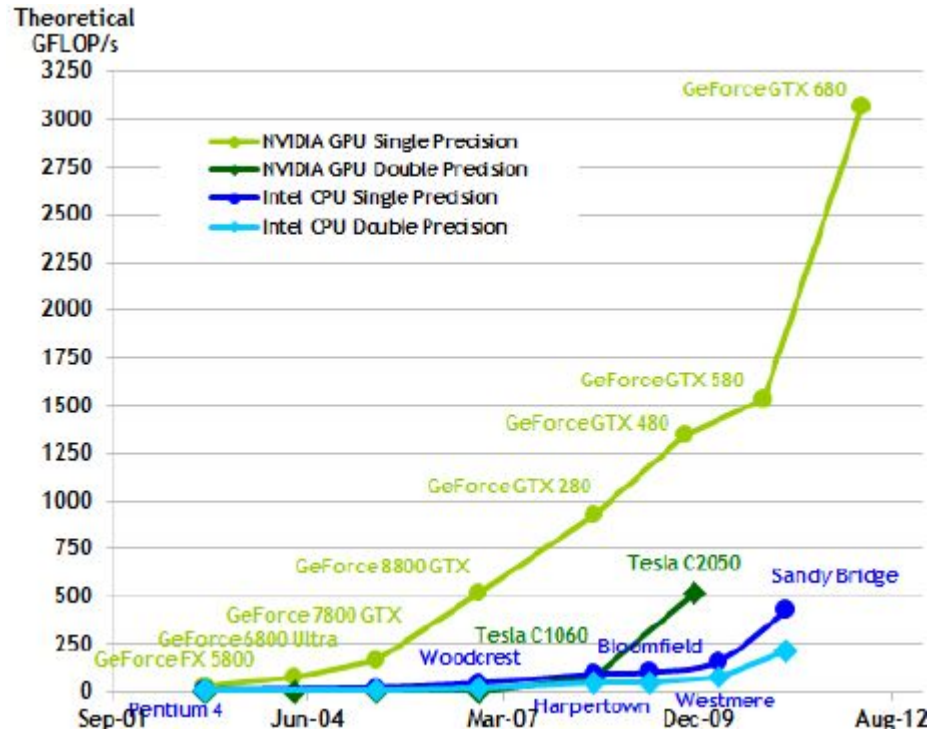
Nvidia GeForce 256

There are different kinds of GPUs with different purposes.



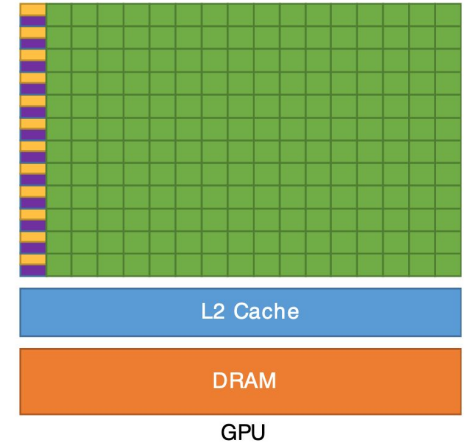
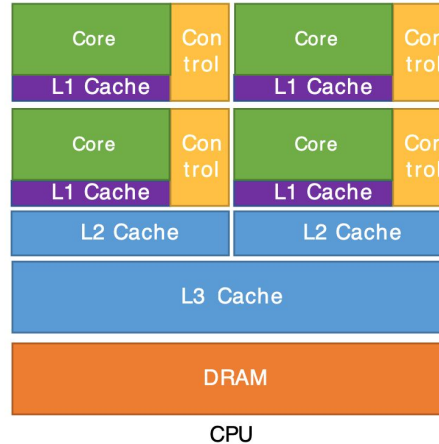
GPUs are optimized for various purposes, with different models excelling in specific precision tasks.

The fastest GPU for single precision computations may not necessarily be the fastest for double precision computations.



GPU VS CPU

- CPUs can handle more complex workflows compared to GPUs.
- CPUs don't have as many arithmetic logic units or floating point units as GPUs.
- CPUs have more cache memory than GPUs.



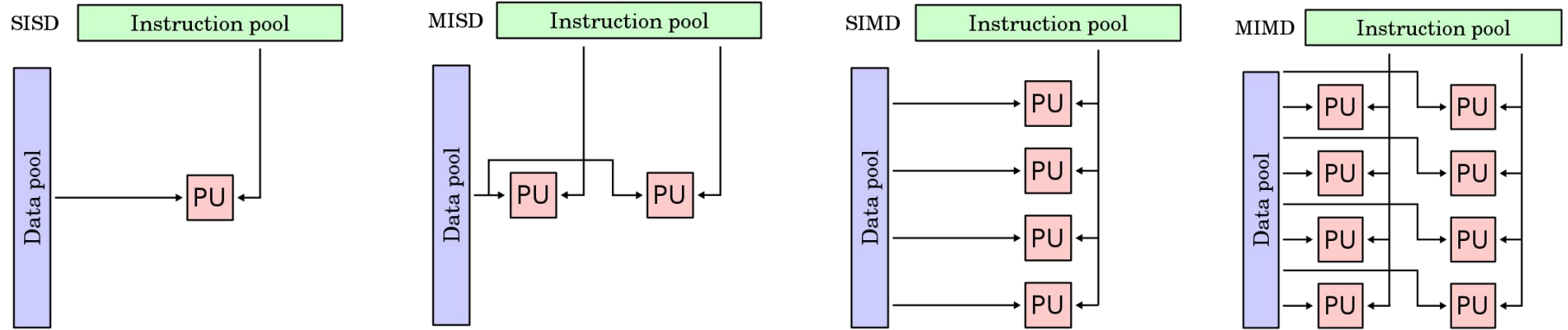
GPU Hardware Model

Key Design Features:

- Many-Core Design
 - 16000 cores - RTX 4090
- High memory bandwidth
 - 1008 GB/s - RTX 4090

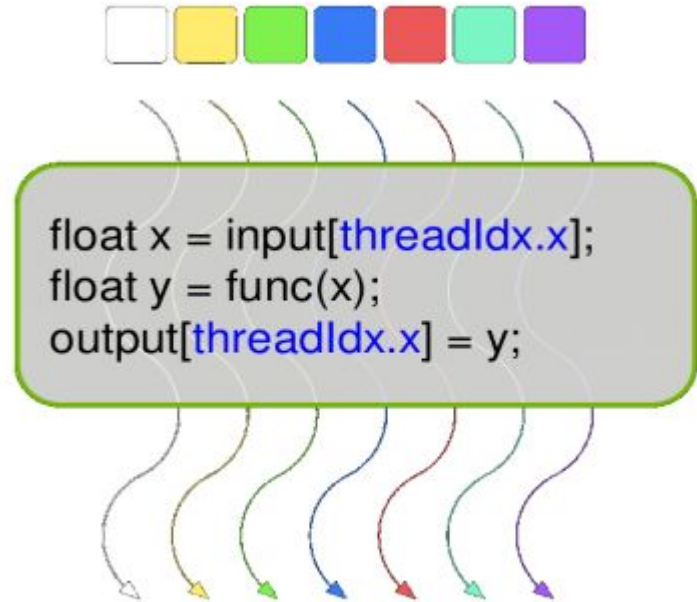


Flynn taxonomy



GPU is an SIMD machine

- A group of threads grouped together(**Warp**), executes the same set of instructions(**Kernel**) in **parallel**.
- Nvidia comes up with SIMT that is a more relaxed version of SIMD.



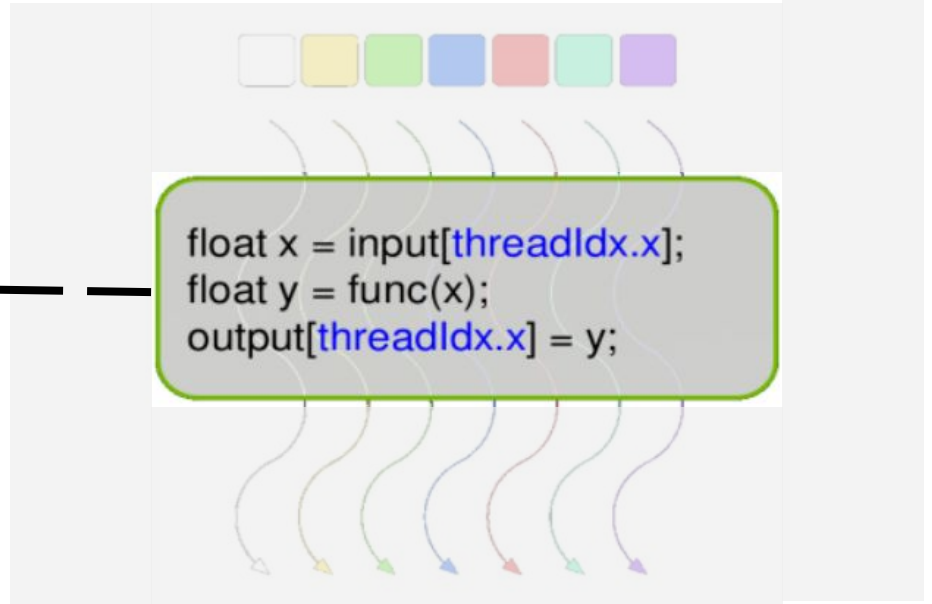
Kernel

In CUDA, a **kernel** is a function designed to **run in parallel** on a GPU.

Kernel

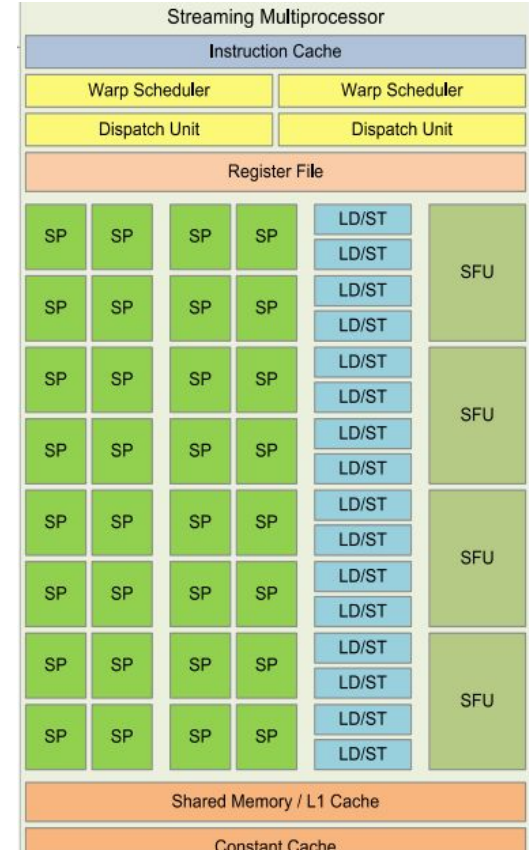


```
float x = input[threadIdx.x];  
float y = func(x);  
output[threadIdx.x] = y;
```



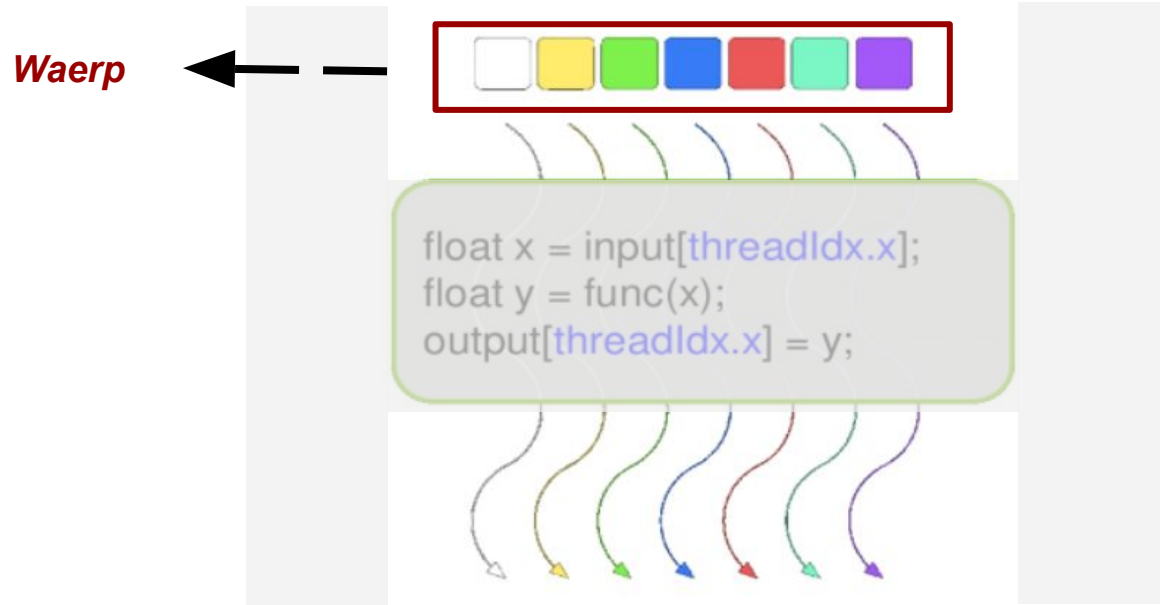
Streaming multiprocessors

On the GPU, a **kernel** call is executed by one or more **streaming multiprocessors**, or **SMs**. The SMs are the hardware homes of the CUDA cores that execute the threads.



Warp

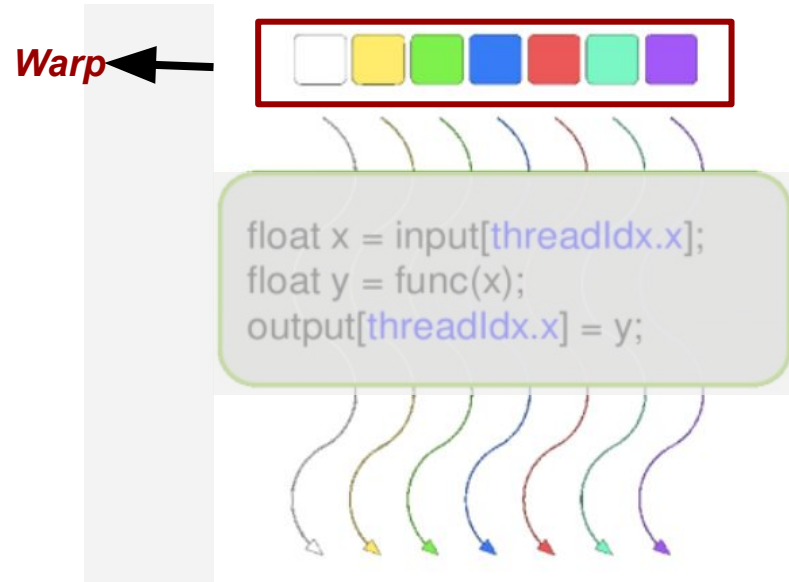
Warps are the set of 32 threads that executes the same kernel.



Warp Scheduling

In GPU architecture, the Streaming Multiprocessors (SMs) are responsible for determining which warp (a group of threads) will execute on which cores.

Scheduling warps is managed by the schedulers within the SMs.



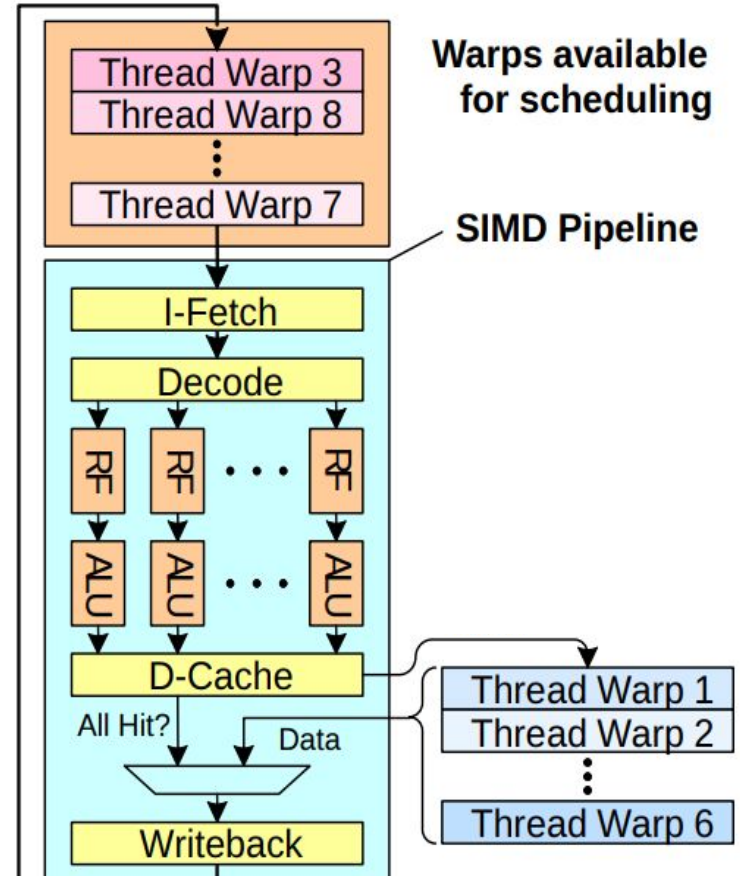
Warp Scheduling (con't)

Scheduler fetches one or two instructions from a warp to execute.

SM (Streaming Multiprocessor) schedulers operate dynamically.

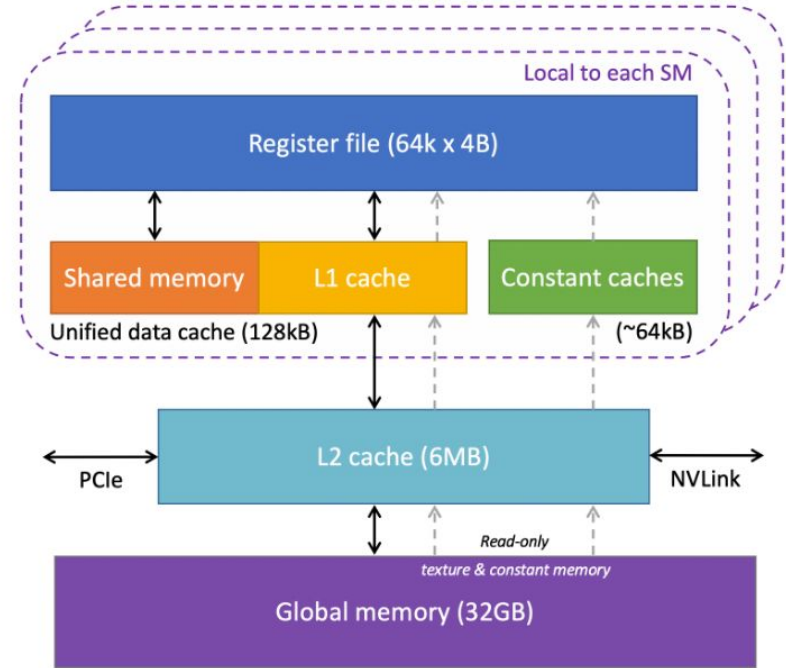
+ Why not Static Warp Assignment?

=> Waiting until all assigned warps are finished.



GPU Memory Structure

Similar to CPUs, GPUs have hierarchical memory structure and multiple caches are applied to different layers.



GPU Memory Structure (con't)

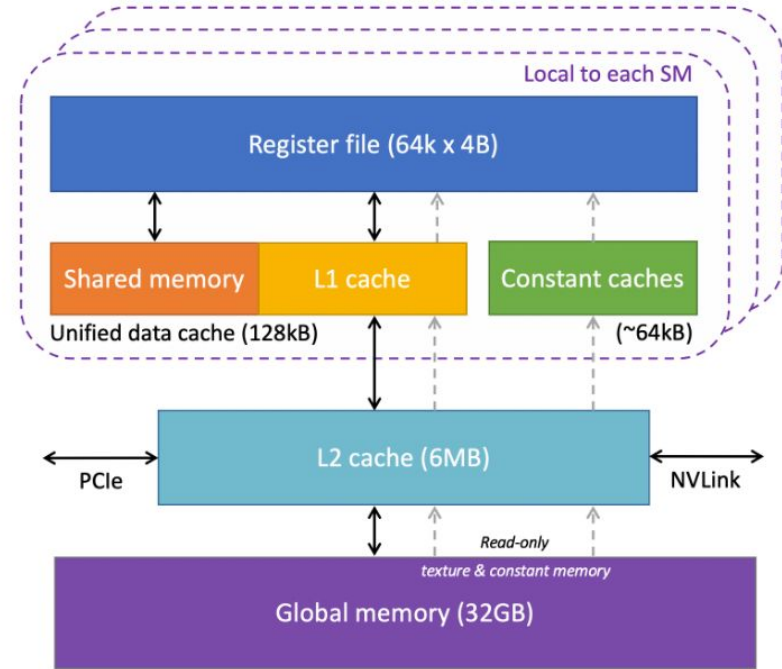
Shared Memory:

- Used for communication between CUDA cores within the same block.
- Enables fast data exchange and synchronization among threads.

Constant Cache:

- Utilized for broadcasting constant variables to all threads within a block.
- Enhances efficiency by providing quick access to read-only data shared across threads.

Why not Static Warp Assignment?



GPU memory levels and sizes for the NVIDIA Tesla V100.

Warp Scheduling (con't)

Think about this example:

The following kernel is assigned to a warp:

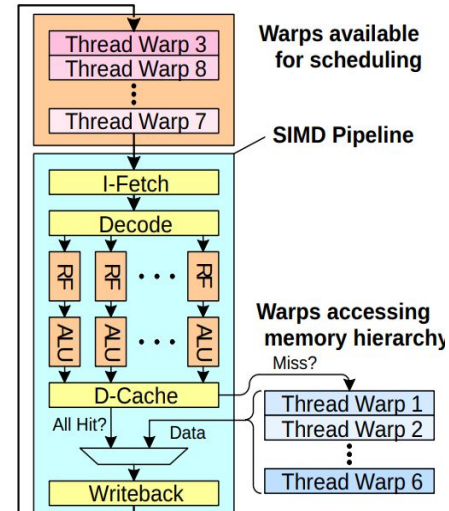
What happen if SM does not have **Mem** data in it registers or caches?

```
fetch_from_mem = Mem[thread_id]  
calc = fetch_from_memory * 2
```

Warp Scheduling (con't)

By utilizing Warp Scheduling, memory latency is concealed. This means the scheduler can swap the current warp with another warp and resume execution of the original warp once the necessary data becomes available in the Streaming Multiprocessor (SM).

```
fetch_from_mem = Mem[thread_id]
calc = fetch_from_memory * 2
```



Introduction to CUDA

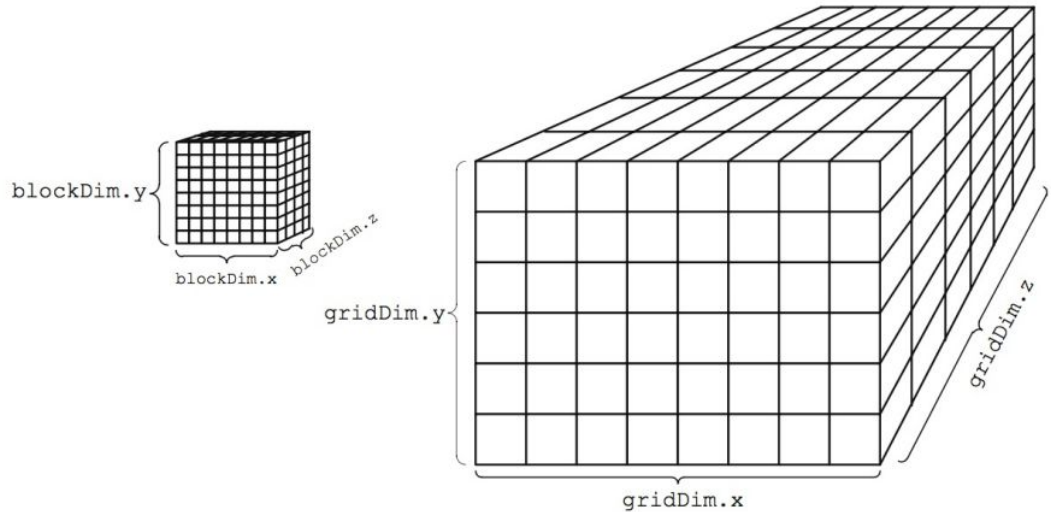
CUDA stands for **Compute Unified Device Architecture**.

CUDA is a **parallel computing platform** and **programming model** created by NVIDIA.

Programmer View

Each Kernel is described by 2 variable:

- 1) Block Dim
- 2) Grid Dim



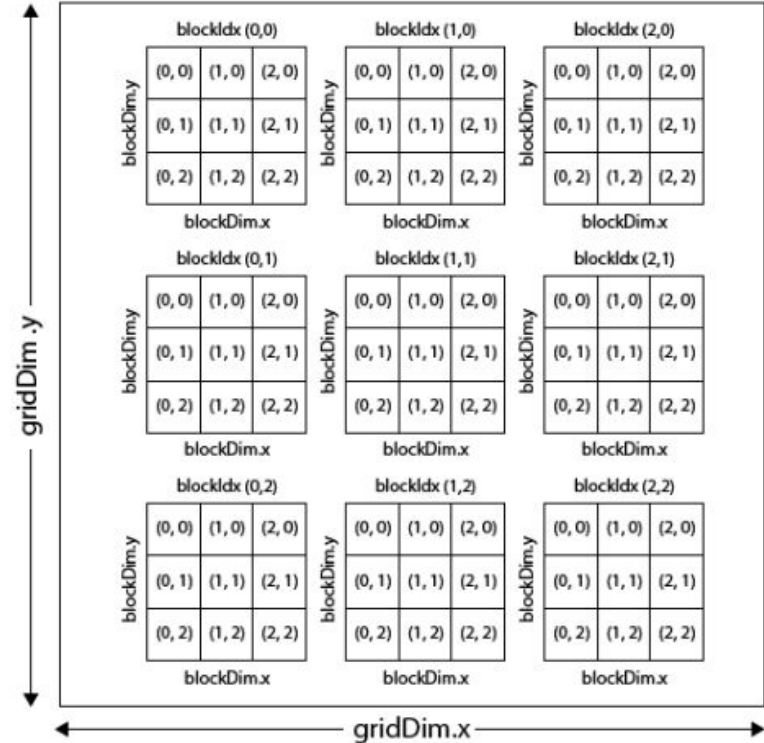
Programmer View

Programmer describe the execution flow of the program using A Grid of Blocks.

Benefits of this model:

- 1) Better Description of multi-dim application (Image Processing)
- 2) Make Hardware mapping easier

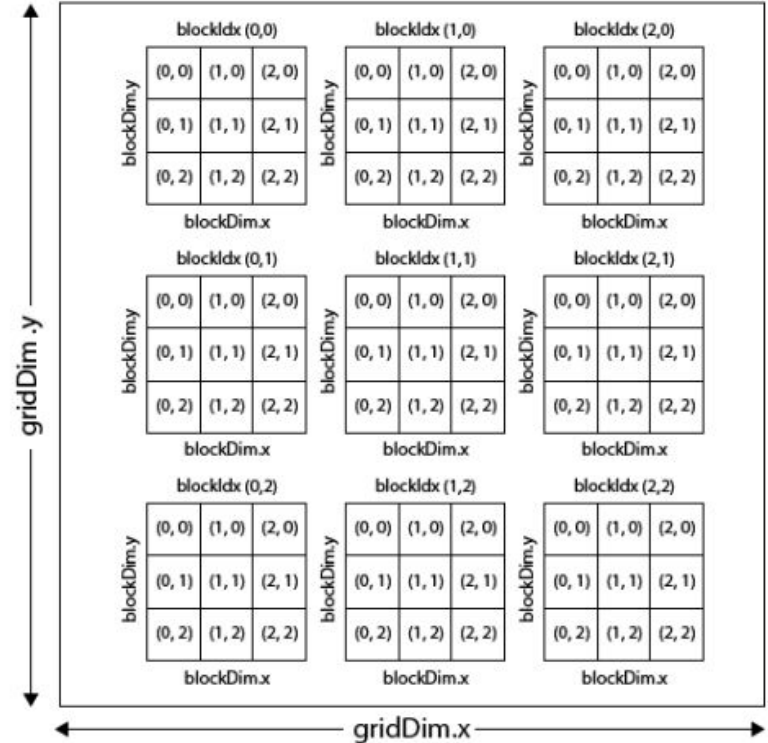
CUDA Grid



Kernel Variables

- threadIdx.x
- threadIdx.y
- threadIdx.z
- blockIdx.x
- blockIdx.y
- blockIdx.z
- blockDim.x
- blockDim.y
- blockDim.z

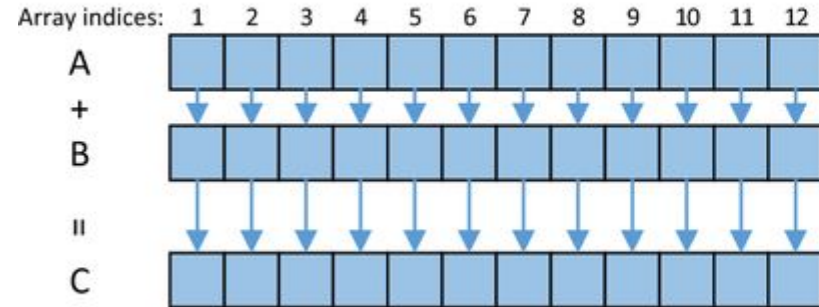
CUDA Grid



Vector Addition Example

What are the dim variables of Grid and Block?

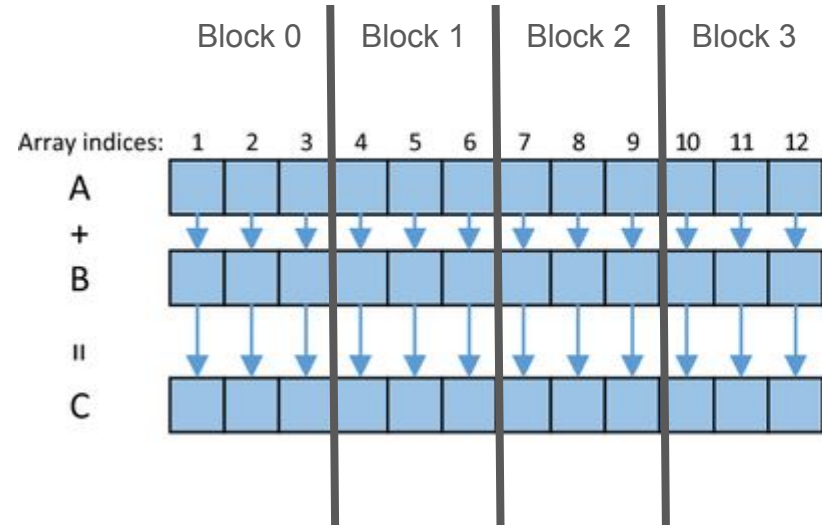
What is the job of each thread?



Vector Addition Example

What are the dim variables of Grid and Block?

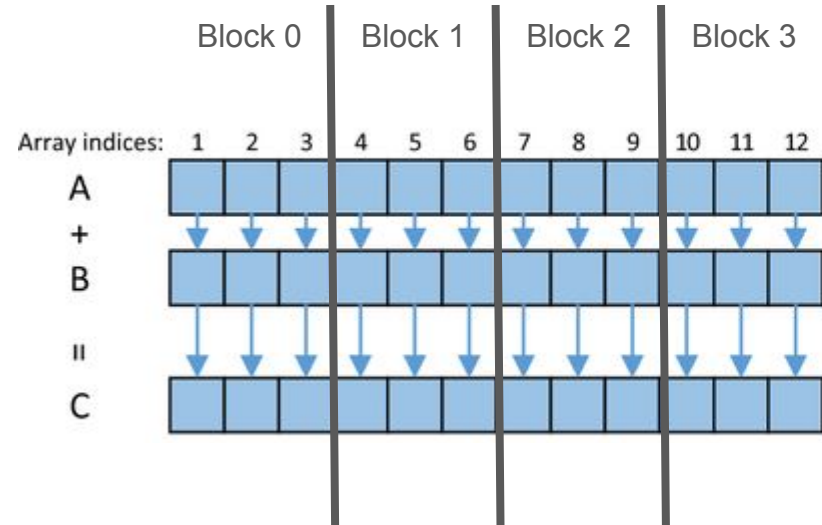
What is the job of each thread?



Vector Addition Example

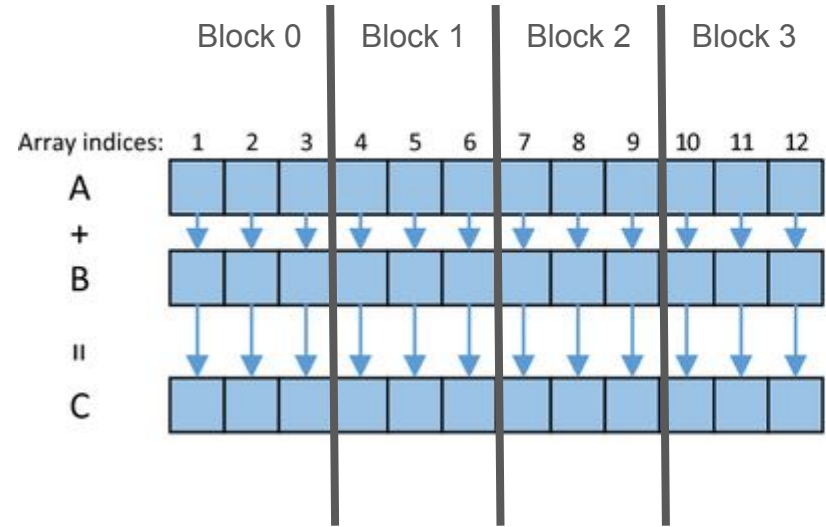
How to describe the task of each thread when all of them executing the same kernel?

- + Use Block and Thread dim for indexing



Vector Addition Example

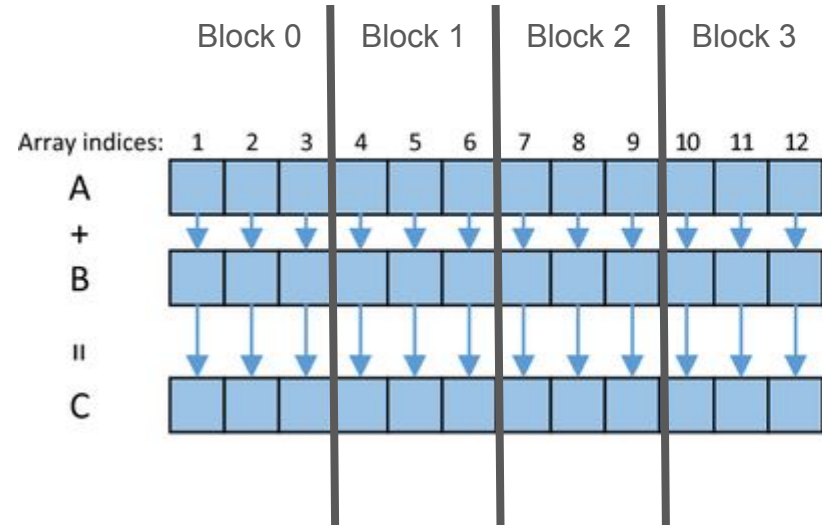
What should thread in Block 2 with Id 2 do?



Vector Addition Example

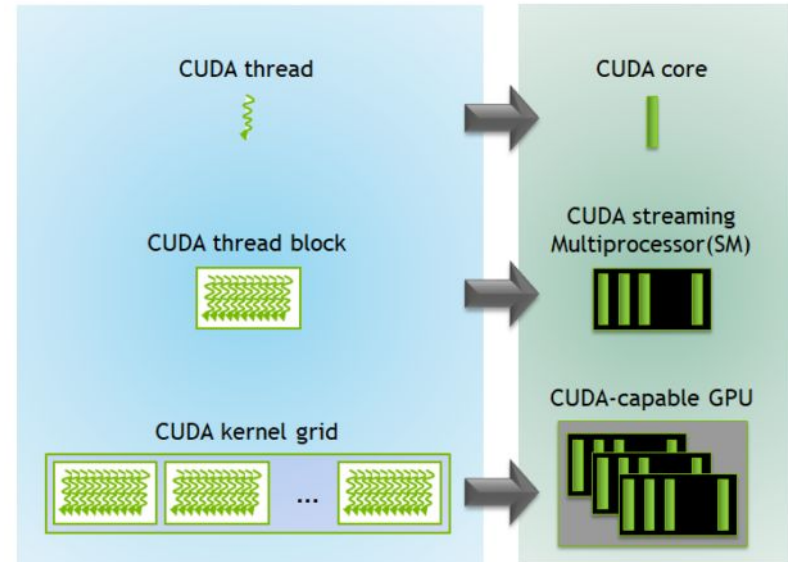
What should thread in Block 2 with Id 2 do?

$INDEX = blockIdx.x * blockDim.x + threadIdx.x$
 $C[INDEX] = A[INDEX] + B[INDEX]$



Hardware Mapping

- Each block should be executed exclusively on one Streaming Multiprocessor (SM).
- The block is subdivided into warps, which are then scheduled within the SM.



What We Are Going to Do on next Session

- Learn about the compilation flow of a cuda program
- Use CUDA to program a parallel programs.
- Compare CUDA and CPU results